

BAT Engineering

Hose Management System (HMS) 2.0

Technical Design Document

System architecture, data model, API, offline sync, security and delivery

Version	1.0
Date	27 June 2026
Status	For review
Classification	Confidential
Prepared for	BAT Engineering
Prepared by	HF Technologies - Solution Architecture

1 Document control

Version	Date	Author	Summary of change
0.1	Jun 2026	Architecture	Initial System Design (draft, markdown)
0.2	Jun 2026	Architecture	Mobile & Offline Addendum A (draft, markdown)
1.0	27 June 2026	Architecture	Consolidated Technical Design Document; review refinements incorporated

Supersession. This document consolidates and supersedes the earlier System Design and Mobile/Offline Addendum drafts. The field application described in those drafts as an offline *PWA* is superseded: the field client is delivered as a **packaged mobile app** built with Ionic + Angular + Capacitor (Angular running in a native shell), as defined herein. No competing approaches remain.

Contents

1 Document control	2
2 Introduction	5
2.1 Purpose	5
2.2 Scope	5
2.3 Definitions	5
2.4 References	5
3 Problem statement and objectives	5
4 Architecture overview	6
4.1 Guiding principles	6
4.2 Logical architecture	6
4.3 Client topology	6
5 Technology stack	7
5.1 Backend and data	7
5.2 Frontend and mobile	7
5.3 Infrastructure and operations	7
6 Domain and data model	8
6.1 Core entities	8
6.2 Asset and inspection lifecycle	8
6.3 Integrity and history rules	8
6.4 Sync- and migration-readiness columns	8
6.5 Change feed (SyncChange)	9
6.6 Idempotency table	9
6.7 Device registry	9
6.8 Entity relationships	9
7 API design	9
7.1 Conventions	9
7.2 Core resource endpoints	9
7.3 Sync endpoints	9
8 Offline-first mobile design	10
8.1 Packaged-app model	10
8.2 First-launch bootstrap (online)	10

8.3 Offline sessions and the offline window	10
8.4 Sync triggers	10
8.5 On-device store	10
8.6 Sync engine	10
9 Conflict-resolution policy	11
10 Security and access control	11
10.1 Authentication	11
10.2 Authorisation	11
10.3 Offline authentication and device governance	11
10.4 Audit and compliance ledger	11
10.5 Data protection	11
11 Asynchronous processing and eventing	12
12 Observability and SRE	12
13 Infrastructure and deployment	12
14 Data migration approach	12
15 Build versus buy: sync engine	12
16 Implementation sequence	12
17 Risks, decisions and open questions	13
17.1 Key decisions	13
17.2 Open questions	13

2 Introduction

2.1 Purpose

This document specifies the technical design for HMS 2.0, a clean greenfield rebuild of the BAT Engineering Hose Management System. It defines the architecture, technology stack, data model, API surface, the offline-first mobile design, security and access control, infrastructure, the migration approach, and the delivery sequence. It is the authoritative engineering reference and is read alongside the Functional Specification Document.

2.2 Scope

HMS 2.0 covers customer, asset, product, reference-data, inspection, certificate and retest-scheduling management, exposed through a web staff console and a packaged mobile inspector application, on a Python/FastAPI backend with PostgreSQL. The first production release (v1) is deliberately narrow: the web staff/admin console, the offline mobile inspector app, and the sync-ready backend foundation. A responsive customer self-service portal is included as web only; a dedicated customer mobile app, advanced analytics, external integrations and AI-assisted features are designed for but scheduled to later phases.

2.3 Definitions

Term	Meaning
Packaged mobile app	An Angular application running inside a native iOS/Android shell via Capacitor. It is a real installable app with native plugin access, but the UI layer is web technology - not a hand-written Swift/Kotlin native build.
Asset	A physical hose record, the central entity, with two configured ends (A and B), a product, a nominal bore and a lifecycle state.
WP / TP	Working pressure / test pressure, defined per product per nominal bore.
Grave / condemned date	The end-of-life date after which a hose must be withdrawn from service.
Certificate	An issued, versioned, verifiable test certificate for an asset, produced from an approved inspection.
Tombstone	A soft-deleted record retained so the deletion can propagate to offline devices during sync.
Sync scope	The subset of data a given user/device is entitled to hold offline (global reference data plus their assigned customers/assets).

2.4 References

- Functional Specification Document - BAT HMS 2.0 (companion document).
- Legacy HMS reference audit (used only to capture fields, columns, workflows and product categories; not scraped for data).

3 Problem statement and objectives

The legacy HMS is functional but structurally limited. It is a server-rendered jQuery 1.10.2 application; reference data (couplings, materials, attach methods) is stored as free text; records are hard-deleted with no audit trail; user passwords appear in editable grids; there is no field/offline workflow; and certificates are static PDFs with no verification. Because hoses are pressure-bearing, safety-critical assets and their certificates act as compliance evidence, these are material risks, not cosmetic ones.

The objectives of HMS 2.0 are: a clean normalised domain model with referential integrity; an immutable, tamper-evident audit and compliance ledger; full retention of safety/compliance history (no destructive deletes); an offline-capable mobile inspection workflow; verifiable digital certificates; proactive retest scheduling; and a modern, typed API consumed by an Angular web console and a packaged mobile app - all on the existing OCI/Kubernetes/Terraform platform.

4 Architecture overview

4.1 Guiding principles

- **Clean rebuild, migrate later.** Build greenfield against synthetic (deliberately dirty) and sanitized sample data; move legacy data through a controlled ETL later.
- **Sync-ready and migration-ready from day one.** The schema carries optimistic-concurrency, soft-delete/tombstone and provenance columns and a change feed from the first commit, so neither offline sync nor migration is a costly retrofit.
- **Modular monolith first.** One FastAPI deployable with clear module seams; only the certificate and rules engines are separated (over gRPC). Split further only when scale demands it.
- **Honest capabilities.** Background sync is best-effort; the field client is a packaged app; Kafka is staged behind a transactional outbox.

4.2 Logical architecture

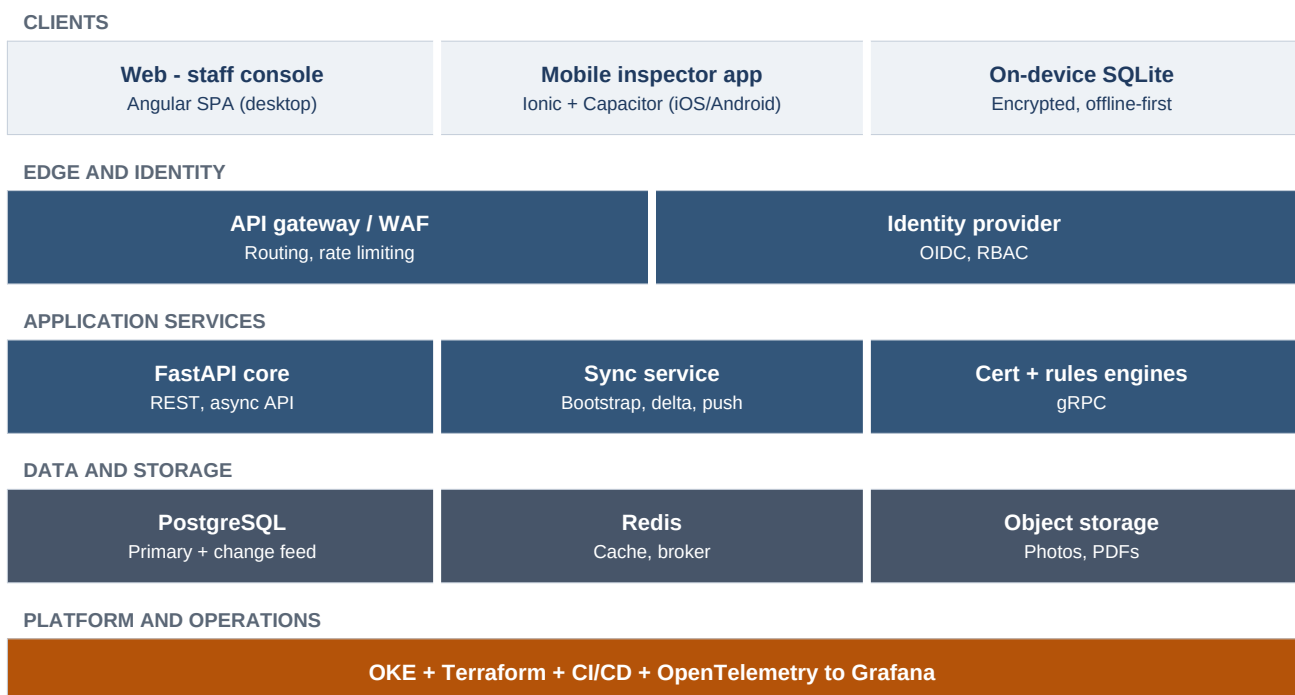


Figure 1 - Logical architecture. Colour groups client surfaces, backend services and data/platform components. The field client is a packaged mobile app (Angular in a Capacitor shell), not a separate native build.

4.3 Client topology

Surface	Form	Offline	In v1
Staff / admin console	Web (Angular SPA)	No - desktop, data-grid heavy	Yes
Inspector app	Packaged mobile (Ionic + Capacitor, iOS/Android)	Yes - offline-first	Yes
Customer self-service portal	Responsive web (Angular)	No (online)	Yes (web only)
Customer mobile app	Packaged mobile	Online-first	No - later phase

All surfaces live in one Nx workspace sharing a UI/design-system library and TypeScript domain types generated from the FastAPI OpenAPI specification, eliminating duplicated contracts.

5 Technology stack

5.1 Backend and data

Concern	Choice	Rationale
API framework	FastAPI (Python 3.12)	Async, first-class OpenAPI, Pydantic validation; typed contract for clients
Validation	Pydantic v2	Request/response models, settings, strict typing
ORM / migrations	SQLAlchemy 2.0 (async) + Alembic	Typed 2.0 API, async, versioned migrations
Primary database	PostgreSQL 16 (OCI managed)	Relational integrity, JSONB, full-text search, partial indexes
Cache / broker	Redis	Caching, Celery broker, rate limiting
Background jobs	Celery	PDF generation, reminders, bulk operations, imports
Internal services	gRPC	Typed, low-latency internal contracts for cert + rules engines
Eventing	Kafka (staged)	Audit/integration stream; introduced behind an outbox when justified
PDF generation	WeasyPrint (HTML/CSS to PDF/A)	Maintainable, styled certificates
Auth	OIDC (Keycloak or OCI Identity Domains)	Standards-based; staff + customer realms

5.2 Frontend and mobile

Recommendation: Ionic + Angular + Capacitor. It is the only option that reuses the Angular codebase, the Nx workspace, the design system and the generated API client, while compiling one TypeScript/Angular codebase to both iOS and Android. The output is a **packaged app**: a native shell wrapping the Angular UI with native plugin access (SQLite, camera, geolocation, secure storage, network). For a forms-, photo- and data-entry workflow this is more than adequate; it is explicitly not a hand-written Swift/Kotlin native build, and should not be described as such.

Option	Local store	Reuse with web	Verdict
Ionic + Angular + Capacitor	SQLite (+ SQLCipher)	High (Angular, Nx, design system, API client)	Recommended
Flutter	sqflite / drift	None (Dart, separate codebase)	Strong tech, second stack to own
React Native	op-sqlite / quick-sqlite	Partial (TS, but React)	Viable; loses Angular reuse
.NET MAUI	sqlite-net-pcl	None (C#)	Off-stack
Native (Kotlin + Swift)	Room + GRDB	None (two codebases)	Best fidelity, highest cost

Data grids (asset, product, certificate) use AG Grid or PrimeNG for server-side filtering, sorting, virtual scroll and column configuration.

5.3 Infrastructure and operations

Concern	Implementation
Compute	OCI Kubernetes Engine (OKE); separate node pools for API, workers, gRPC services
Registry	OCI Container Registry (OCIR)
Database	OCI managed PostgreSQL (HA, automated backups, PITR)
Object storage	OCI Object Storage (S3-compatible) for PDFs and photos

Concern	Implementation
Ingress	OCI API Gateway + Load Balancer + WAF
Secrets	OCI Vault
IaC / packaging	Terraform modules per environment; Helm charts; optional ArgoCD
CI/CD	GitHub Actions: lint, test, SAST + image scan, build, push OCIR, deploy via Helm
Observability	OpenTelemetry to Prometheus / Loki / Tempo / Grafana
Notifications	OCI Email Delivery (email) + Twilio (SMS)

6 Domain and data model

6.1 Core entities

Entity	Purpose
User, Role, Permission	Identities and RBAC; customer users link to one Customer
Customer, CustomerLocation, CustomerContact	Customer account, its sites and named contacts
Asset	The hose record: customer, location, current product, two end configurations, tag, lifecycle state, manufacture and condemned dates
AssetEndConfiguration	End A / End B build spec referencing coupling, add-on, attach method and material
Product, ProductPressureRating	Catalogue item and its normalised WP/TP-by-bore matrix
Coupling, CouplingAddOn, AttachMethod, Material, NominalBore, Standard	Reference lookups (formerly free text)
InspectionTemplate, InspectionQuestion	Versioned question sets
Inspection, InspectionAnswer, PressureTestResult, InspectionPhoto	Inspection event and its captured data and evidence
Certificate	Issued, versioned, verifiable certificate with verification hash
RetestSchedule, Notification	Computed due dates and outbound reminders
AuditEvent	Append-only, hash-chained change log
SyncChange, IdempotencyKey, Device	Sync change feed, push de-duplication, and device registry (see below)

6.2 Asset and inspection lifecycle

Assets follow an explicit state machine - DRAFT, IN_SERVICE, DUE, OVERDUE, CONDEMNED, RETIRED - so an out-of-life hose can never appear serviceable. Inspections move DRAFT, SUBMITTED, APPROVED or REJECTED; a certificate can be issued only from an APPROVED inspection.

6.3 Integrity and history rules

- No hard deletes on assets, inspections, certificates or customers - they carry a soft-delete tombstone and remain in history.
- Reference data is foreign-key enforced, so assets cannot reference values that do not exist and reports group cleanly.
- Every mutating operation writes an AuditEvent.

6.4 Sync- and migration-readiness columns

Every syncable table carries, from day one: a client-generatable UUID **id** (UUIDv7); a **version** integer for optimistic concurrency; a server **updated_at**; and a **deleted_at** tombstone. These sit alongside migration provenance columns (legacy identifiers) so the same schema is simultaneously offline-ready and migration-ready.

6.5 Change feed (SyncChange)

Delta sync is driven by a monotonic change feed, not by timestamp filtering (which is unsafe under clock skew and concurrent writes). A SyncChange row - (seq BIGSERIAL, entity, entity_id, op, version, changed_at) - is appended on every write via the transactional outbox. The pull endpoint reads WHERE seq > :cursor ORDER BY seq, which is correct, replayable and resumable; the device's cursor is simply the last seq it applied.

6.6 Idempotency table

Pushed operations carry an idempotency key. An IdempotencyKey table records keys and their prior results, so a replayed push is a safe no-op returning the original outcome.

6.7 Device registry

Offline trust is governed server-side, not by the client alone. A **Device** table records device and user identity, last sync time, a revoked flag, the permitted offline window, app version and platform:

Column	Purpose
device_id (UUID)	Stable per-installation identifier
user_id	Owning user
platform / app_version	iOS or Android, and installed app version (for support and forced-upgrade policy)
last_sync_at	Last successful sync; basis for offline-window enforcement
offline_window_days	Server-set maximum offline duration before re-authentication is forced
revoked	When set, the device is denied at next sync and instructed to wipe local data

6.8 Entity relationships

- Customer has many CustomerLocations, CustomerContacts, Assets and (portal) Users.
- Asset references one Product and two AssetEndConfigurations, and has many Inspections.
- Product has many ProductPressureRatings (one per nominal bore).
- Inspection has many InspectionAnswers and InspectionPhotos, one optional PressureTestResult, and issues one Certificate.
- User owns many Devices; every write emits one AuditEvent and one SyncChange.

7 API design

7.1 Conventions

A versioned, resource-oriented REST API under /api/v1, auto-documented via OpenAPI. Plural nouns; nesting where ownership is real; cursor or offset pagination; consistent filter/sort/page parameters; RFC 7807 problem-details for errors; ETag/optimistic-concurrency on multi-editor records. Long-running operations return 202 with a job id and report progress.

7.2 Core resource endpoints

Representative endpoints: POST /auth/token; GET /customers; GET /customers/{id}/assets; POST /assets; PATCH /assets/{id}; POST /assets/{id}/inspections; POST /inspections/{id}/submit; POST /inspections/{id}/approve; POST /inspections/{id}/certificate; POST /certificates/bulk; GET /products; GET /tests-due; GET /verify/{token} (public); POST /imports/assets.

7.3 Sync endpoints

Endpoint	Purpose
GET /api/v1/sync/bootstrap	First-launch snapshot. Returns the device's full in-scope dataset (assigned customers, assets, locations, reference data, templates) plus the starting SyncChange seq cursor. Used once at provisioning so the first sync is clean rather than a large delta replay.
GET /api/v1/sync/changes?since={cursor}	Delta pull. Ordered upserts and tombstones for in-scope entities since the cursor, plus a new cursor.
POST /api/v1/sync/operations	Push. Batched outbox operations with idempotency keys; applied with version checks; returns per-operation outcome and canonical versions.
POST /api/v1/uploads/presign	Pre-signed URL for direct photo upload to Object Storage.

8 Offline-first mobile design

8.1 Packaged-app model

The inspector app is an Angular application packaged with Capacitor into installable iOS and Android binaries, using native plugins for SQLite (with SQLCipher), secure storage, camera, geolocation and network detection. It is suited to inspection forms, photos, QR scanning, pressure-test capture, data entry and sync.

8.2 First-launch bootstrap (online)

- Require connectivity; authenticate via OIDC (Authorization Code + PKCE); store tokens in secure storage (Keychain/Keystore).
- Register the device (Device table) and create the encrypted local SQLite database, key held in the keystore and unlocked by biometric/passcode.
- Call GET /sync/bootstrap to load the in-scope snapshot and the starting cursor; mark the device provisioned.

8.3 Offline sessions and the offline window

From the second launch the app trusts a cached, bounded offline session. The offline window (for example 7-14 days since last successful online auth/sync) is set server-side per device and enforced on the client; beyond it, online re-authentication is forced. The encrypted database is unlocked by biometric/passcode. Token refresh and revocation re-checks occur when connectivity returns.

8.4 Sync triggers

The product promise is explicit and modest: sync **on app open**, sync **on reconnect** (network listener), and a **manual Sync button**. Periodic OS background sync is best-effort only - iOS and Android may delay or block background work - and is never relied upon for correctness.

8.5 On-device store

SQLite mirrors the in-scope subset. Each row carries sync metadata (server_version, updated_at, sync_state, dirty). An operation outbox queues local mutations - (op_id, entity, entity_id, op_type, payload, idempotency_key, attempts, status). A media table tracks photo files and their upload state. Reference data is pull-only (read-only on device).

8.6 Sync engine

Incremental sync runs push then pull. Push drains the outbox with idempotency keys and applies version checks server-side. Pull applies ordered upserts and tombstones in a transaction and advances the cursor only on success, so a partial sync never loses changes. Photos upload directly to Object Storage via pre-signed URLs, then their metadata is pushed. Retries use exponential backoff.

9 Conflict-resolution policy

The common cases are conflict-free by construction; the residual surface is small and handled explicitly. Last-write-wins is permitted only for harmless draft-only fields.

Data class	Policy
Reference / catalogue data	Server authoritative, pull-only on device - conflicts are impossible.
Inspections assigned to one inspector	Owned by a single inspector; effectively conflict-free in normal use.
Submitted or approved inspections	Locked and server-authoritative; client cannot override.
Asset safety fields	Optimistic concurrency by version; on mismatch a conflict screen is shown. Never silently overwritten.
Draft-only harmless fields	Last-write-wins acceptable (e.g. an unsubmitted local note).

10 Security and access control

10.1 Authentication

OIDC issues short-lived access tokens and refresh tokens. Passwords are hashed with Argon2; multi-factor is supported. Passwords are never displayed in any interface.

10.2 Authorisation

Role	Capability
SUPER_ADMIN	Everything, including user and role administration
HMS_ADMIN	Customers, products, reference data, scheduling
INSPECTOR / ASSEMBLY	Create and edit inspections; capture results in the field
REVIEWER	Approve or reject inspections; issue certificates
CUSTOMER_USER	Row-level scoped to their own customer; read fleet, download certificates

Customer scoping is enforced in the query layer; a customer user cannot retrieve another customer's data. gRPC services use mutual TLS.

10.3 Offline authentication and device governance

Offline access is bounded and revocable. The Device registry is the server-side control point: on reconnect the app refreshes its token and re-checks revocation; a revoked or disabled device/user is rejected at the next sync and instructed to wipe local data and re-authenticate. Client-side window enforcement is a convenience layer on top of this server authority, not a substitute for it.

10.4 Audit and compliance ledger

A dedicated append-only AuditEvent table records actor, action, entity, before/after, timestamp, and a hash chained to the previous row, making retroactive tampering detectable. Certificates carry a verification hash exposed through the public verify endpoint.

10.5 Data protection

Encrypted SQLite on device (SQLCipher), key in the device keystore. Secrets in OCI Vault. TLS in transit; encryption at rest on database and object storage. WAF and rate limiting at the gateway. Short-TTL pre-signed URLs for media. Dependency and image scanning in CI. Sync scope-limiting also minimises the data held on any device.

11 Asynchronous processing and eventing

Celery (Redis broker) runs bulk certificate generation, scheduled retest-reminder runs, escalations, imports/exports and photo processing; jobs are idempotent and report progress. Domain events are written to a transactional outbox in the same transaction as the business change. Kafka is introduced only when external integrations or multi-sink audit streaming justify it; the outbox relay can then publish to Kafka with no change to business code. The certificate engine and rules engine are internal gRPC services - the former isolates CPU-bound PDF rendering and signing, the latter is a reusable validation service.

12 Observability and SRE

One OpenTelemetry pipeline from the API, workers and gRPC services into Prometheus (metrics), Loki (logs) and Tempo (traces), surfaced in Grafana. Dashboards cover API latency and error rates, job queue depth and success, certificate-generation time, reminder delivery and database health. SLOs target API availability and p95 latency, background-job success rate and certificate-generation time. Logs are structured JSON with a correlation/trace id threaded from gateway through workers; Alertmanager drives on-call.

13 Infrastructure and deployment

All services run on OKE, provisioned by Terraform and shipped by GitHub Actions (build, test, scan, push to OCIR, deploy via Helm). Environments are dev, staging and prod from identical IaC, promoted by tag. Rollouts are rolling or blue-green; database migrations are gated and reversible. Managed PostgreSQL provides HA, automated backups and point-in-time recovery; Object Storage holds media; OCI Vault holds secrets.

14 Data migration approach

Given the modest data size, a clean rebuild with a one-time, validated ETL is lower-risk than a strangler proxy. The schema is designed for migration from day one (provenance columns, normalised lookups). The sequence is: audit the legacy data; build extractors into a staging area; transform and de-duplicate free-text reference values into lookups (the most labour-intensive step, with a human-reviewed mapping sheet); load via the bulk-import path with validation and a dry-run mode; reconcile counts and integrity and produce a report; run legacy read-only in parallel for validation; then cut over, retaining the legacy database as an archived fallback. Development proceeds against synthetic, deliberately dirty data plus a small sanitized sample, so the importer and validation are built against realistic mess rather than retrofitted.

15 Build versus buy: sync engine

The default is to build a custom delta-sync over the existing REST API and the SyncChange feed - full control, a natural fit for PostgreSQL + FastAPI + a bounded scope, and no new infrastructure. Before committing, run a short spike on **PowerSync** (Postgres-native offline sync with a Capacitor/JS SDK), which could remove a meaningful share of sync-engine work if it covers the scope and conflict model cleanly. ElectricSQL is a further option; document stores such as CouchDB/PouchDB are a poor fit for this relational model.

16 Implementation sequence

Phase	Outcome
0 - Foundations	Schema with version, updated_at, deleted_at, SyncChange, IdempotencyKey and Device tables; OKE/Terraform bootstrap; CI/CD skeleton; OIDC + RBAC; Nx workspace for web and Capacitor targets
1 - Web HMS (core)	Customers, locations, contacts; products + pressure matrix; reference-data admin; assets with normalised ends; search - built on synthetic/dirty data; migration dry-run

Phase	Outcome
2 - Inspections and certificates	Inspection workflow (new asset / service), templates, pressure-test capture, review/approve, signed PDF/A certificates (single and bulk), audit ledger
3 - Offline inspector mobile app	Packaged Ionic/Capacitor app (iOS + Android); encrypted on-device SQLite + outbox; bootstrap; offline session and Device governance; delta-sync engine; QR scanning; offline photo capture
4 - Portal, analytics, integrations	Responsive customer portal; fleet analytics; integration API and webhooks; standards validation engine; Kafka if/when integrations warrant; customer mobile app evaluation

Security, audit and observability are threaded through every phase. v1 is Phases 0-2 plus Phase 3; the customer mobile app and external integrations are explicitly deferred.

17 Risks, decisions and open questions

17.1 Key decisions

Decision	Choice	Trade-off
App topology	Modular monolith + 2 gRPC services	Simpler now; split along seams later
Eventing	Outbox-first, Kafka when justified	Avoids day-one Kafka ops cost
Field client	Packaged Ionic/Capacitor app	Code reuse and one stack; not hand-native
Sync model	Custom delta-sync (spike PowerSync)	Control and fit vs build effort
Deletes	Soft-delete + tombstones everywhere	Slightly more query complexity for compliance
v1 scope	Web console + inspector app + foundation	Narrow first release reduces risk

17.2 Open questions

- Offline-window length per device (field reality versus security).
- Strictness of asset-field conflict handling (auto-resolve non-safety fields, or always surface).
- Background-sync expectations versus OS limits (is open/reconnect/manual sufficient).
- Build versus buy the sync engine (PowerSync spike outcome).
- Biometric requirement and MDM distribution (Intune/Jamf) assumptions.
- Whether the rules engine should warn or block on invalid hose configurations.
- Photo volume and on-device retention before purge.